# Introduction to Object-Oriented Programming (OOP)

TP2: Encapsulation, `private`/`public`, and Getters/Setters
Practical Session Guide

## 1 Part 1: Theoretical Background

### 1.1 1. The Problem: Exposed Attributes (as in TP1)

Recall the TP1 code:

```
student1.id = -999;       // No one prevents this!
student1.name = null;     // Nor this!
```

Listing 1: TP1 – Exposed attributes

Any external code can change the object's data as it wishes, with no protection. This is a **risk** in real-world programs.

### 1.2 2. The Solution: Encapsulation

**Encapsulation** means hiding an object's internal data and allowing access to it only through **predefined gateways**.

| Keyword | Meaning |
|---|---|
| `private` | The attribute is visible **only inside the class** |
| `public` | Visible from anywhere |
| **Getter** | A `public` method that *returns* a `private` attribute |
| **Setter** | A `public` method that *modifies* a `private` attribute |

**Golden rule:** all attributes are `private`; all access goes through `public` getters and setters.

### 1.3 3. The Getter / Setter Pattern

```
// Getter (read)
public String getTitle() {
    return title;
}

// Setter (write)
public void setTitle(String title) {
    this.title = title;   // 'this' will be explained in TP3
}
```

Listing 2: Getter and Setter pattern

**Naming convention:**

- Getter: `get` + attribute name with a capital first letter (e.g. `getTitle`).

- Setter: `set` + attribute name with a capital first letter (e.g. `setTitle`).
- For `boolean` attributes, prefer `is` over `get` (e.g. `isAvailable`).

# 2 Part 2: Guided Practical Work (Guided TP) — Eclipse Environment

## 2.1 Step 1: Create the Project

1. Open **Eclipse IDE** and select your Workspace.
2. Go to `File > New > Java Project`, name it `TP2_IntroOOP`, and click **Finish**.

## 2.2 Step 2: Program the Blueprint (Class `Book`) — First Version (Exposed)

1. Inside the `src` folder, create a new class named `Book`.
2. Write the following *exposed* version (as in TP1):

Write this code:

```java
public class Book {

    // Attributes (public by default - NOT recommended)
    String title;
    String author;
    double price;

    // Behavior
    public void displayInfo() {
        System.out.println("Title  : " + title);
        System.out.println("Author : " + author);
        System.out.println("Price  : " + price + " DA");
        System.out.println("----------------------------");
    }
}
```

Listing 3: Book.java (Exposed version)

## 2.3 Step 3: The Main Program (Main Class)

1. Create a new class named `MainApp` and check the option to generate the `public static void main` method.
2. Write the following code to instantiate the objects and expose the problem:

```java
public class MainApp {
    public static void main(String[] args) {

        Book book1 = new Book();
        book1.title  = "Clean Code";
        book1.author = "Robert C. Martin";
        book1.price  = 3200.0;

        // Anyone can break the data!
        book1.price  = -500.0;   // No error -- but nonsense!
        book1.displayInfo();
    }
}
```

Listing 4: MainApp.java (Exposing the problem)

3. Click the **Run** button. Notice that the price is `-500.0 DA` with no objection. **That is the problem.**

## 2.4 Step 4: Convert to an Encapsulated Version

Modify `Book.java` so that all attributes are `private`, and add getters and setters:

```
public class Book {

    // 1. Private Attributes (Encapsulation)
    private String title;
    private String author;
    private double price;

    // 2. Getters (public read access)
    public String getTitle()  { return title; }
    public String getAuthor() { return author; }
    public double getPrice()  { return price; }

    // 3. Setters (public write access)
    public void setTitle(String title)   { this.title  = title; }
    public void setAuthor(String author) { this.author = author; }
    public void setPrice(double price)   { this.price  = price; }

    // 4. Behavior
    public void displayInfo() {
        System.out.println("Title  : " + title);
        System.out.println("Author : " + author);
        System.out.println("Price  : " + price + " DA");
        System.out.println("----------------------------");
    }
}
```

Listing 5: Book.java (Encapsulated version)

**Notice the difference from TP1:** all attributes are now `private`. No external code can write `book.price = -5;` directly anymore.

## 2.5 Step 5: Update `MainApp` to Use Getters and Setters

```
public class MainApp {
    public static void main(String[] args) {

        // A. Create book1 and fill it via SETTERS
        Book book1 = new Book();
        book1.setTitle("Clean Code");
        book1.setAuthor("Robert C. Martin");
        book1.setPrice(3200.0);

        // B. Create book2 and fill it via SETTERS
        Book book2 = new Book();
        book2.setTitle("Effective Java");
        book2.setAuthor("Joshua Bloch");
        book2.setPrice(4500.0);

```

```
16          // C. Display both
17          System.out.println("=== Book 1 ===");
18          book1.displayInfo();
19
20          System.out.println("=== Book 2 ===");
21          book2.displayInfo();
22
23          // D. READ a value via GETTER
24          System.out.println("Book 1 price is: " + book1.getPrice() + " DA
                ");
25
26          // E. MODIFY a value via SETTER
27          book1.setPrice(2900.0);
28          System.out.println("=== Book 1 after price update ===");
29          book1.displayInfo();
30
31          // F. Try to access a private field directly (UNCOMMENT TO SEE
                ERROR)
32          // book1.price = -5.0;    // Compile error: price has private
                access in Book
33      }
34  }
```

Listing 6: MainApp.java (Encapsulated access)

**Key learning moment:** Uncomment the last line `book1.price = -5.0;`. You will see a compile-time error: `price has private access in Book`. This is exactly what encapsulation is meant to do — block direct access and force everyone to go through the setter.

### 2.6 Step 6: Run and Verify

Click **Run**. You should see:

- `Book 1` at `3200.0 DA`.
- `Book 2` at `4500.0 DA`.
- Message: `Book 1 price is:  3200.0 DA` (through the getter).
- `Book 1` at `2900.0 DA` (after the setter).
- No compilation errors.

## 3 Part 3: Individual Tasks

Before pushing your project, complete the following tasks individually within the same project to solidify your understanding.

### Task 1 (Add a New Encapsulated Attribute)

- Add a `private int year` attribute (publication year) to `Book`.
- Add `getYear()` and `setYear(int year)`.
- Update `displayInfo()` so it prints the year.

### Task 2 (A Third Book That Represents You)

- In `MainApp`, create `book3` — your favorite book!

- Fill it entirely through **setters** (no direct access).
- Call `displayInfo()`.

### Task 3 (Explore Encapsulation — Private Access Error)

- In `MainApp`, try (as a comment first):

```
// book1.title = "Hacked!";
```

Listing 7: Task 3 – Try direct access

- Remove the comment and observe the **compile-time error**.
- Write a short **comment** in the code explaining why this error is a **feature**, not a bug.

### Task 4 (Modify a Book's Price via Setter)

- Change `book2`'s price to `3999.0` using `setPrice`.
- Print `book2.displayInfo()` to confirm the change.

### Task 5 (Read All Values via Getters)

- At the end of `MainApp`, manually print `book3`'s full information (without `displayInfo`) using getters only:

```
System.out.println("My favorite book:");
System.out.println("  Title  -> " + book3.getTitle());
System.out.println("  Author -> " + book3.getAuthor());
System.out.println("  Price  -> " + book3.getPrice() + " DA");
System.out.println("  Year   -> " + book3.getYear());
```

Listing 8: Task 5 – Reading via getters

- Write a **comment** explaining: what is the practical benefit of using getters instead of printing the attribute directly? (Hint: control over output format, ability to add logic later, separation of interface from data.)

### Task 6 (Setter to Fix a Value Before Printing)

- Assume you entered a wrong price in `book1` (e.g. `-100.0`).
- Correct it via `setPrice(3200.0)` before calling `displayInfo()`.
- Note in a comment: **in TP3 we will prevent the invalid value at the source inside the setter (validation).**

## 4 Part 4: Submission Assignment (Integration with GitHub)

Once you have completed the tasks above and verified that the program runs without errors, it is time to submit your practical work via GitHub:

1. Open **GitHub Desktop**.
2. Go to `File > Add local repository`, and select your `TP2_IntroOOP` project folder from your Workspace.
3. When the "create a repository" warning appears, click the link and ensure you select **Java** from the **Git ignore** dropdown menu (this is crucial to prevent uploading compiled files).

4. In the **Summary** field, write a clear commit message like: `Complete TP2 with all 6 tasks`.
5. Click the blue **Commit** button.
6. Publish the repository as a **Private Repository** and click **Publish**.
7. Log in to your GitHub account on the web, navigate to `Settings > Collaborators`, and add your professor's GitHub username `SniperDZ-Pro` to grant them access to evaluate your code.